Zellic

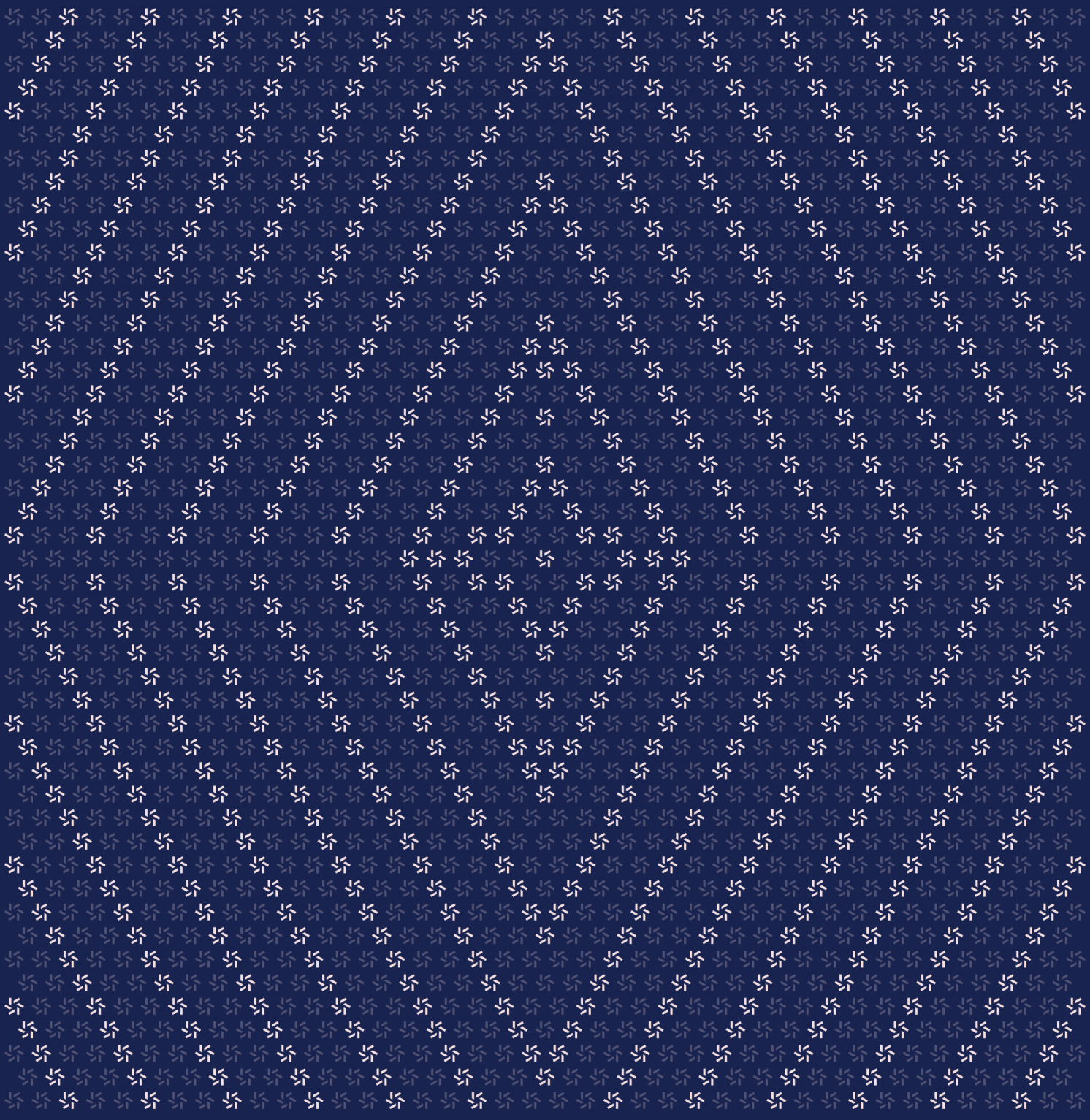
Prepared for
Wyatt Cravatas
Aori

Prepared by
Qingying Jie
Jinseo Kim
Zellic

March 10, 2025

Aori 0.4.0 Upgrade

Smart Contract Security Assessment



Contents

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.

1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Aori from February 24th to March 3rd, 2026. During this engagement, Zellic reviewed Aori 0.4.0 Upgrade's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Can the soft-fail rollback in `AoriSettleLib._settleOrder` leave any partial state behind? Specifically, if the protocol fee accrual via `addNoRevert` fails after filler/`feeRecipient` balances have been written, is the rollback of all three parties (offerer, filler, `feeRecipient`) always complete?
- Are there any reentrancy vectors through hook execution (`HookUtils.executeHook`) that could bypass the EIP-1153 transient storage reentrancy guard — for example, a hook calling back into `fill()` or `withdraw()` during a `deposit()` with hook?
- Are there any deposit or fill paths where a deflationary token could cause accounting inconsistencies that are not caught by the balance-delta check?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

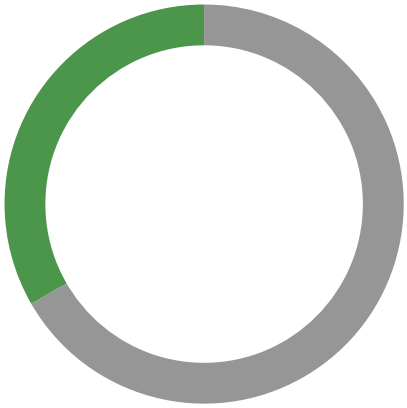
Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Aori 0.4.0 Upgrade contracts, we discovered three findings. No critical issues were found. One finding was of low impact and the other findings were informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	1
Informational	2



2. Introduction

2.1. About Aori 0.4.0 Upgrade

Aori contributed the following description of Aori 0.4.0 Upgrade:

> Aori is designed to securely facilitate peer to peer order execution with trust minimized settlement from any chain to any chain. To accomplish this, Aori uses a combination of off-chain infrastructure, on-chain settlement contracts, and Layer Zero messaging. Solvers expose a simple API to ingest and process order-flow directly to their trading system. The Aori smart contracts ensure that the user's intents are satisfied by the Solver on the destination chain according to the parameters of an intent on the source chain, signed by the user.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

Aori 0.4.0 Upgrade Contracts

Type	Solidity
Platform	EVM-compatible
Target	Only the changes made from 1b3d4bb to b6f5bf4 for the following files
Repository	https://github.com/aori-io/aori ↗
Version	`b6f5bf48f05bcb949ae6a021c855c0e9a770ad45`
Programs	`contracts/*/*.sol`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 1.2 person-weeks. The assessment was conducted by two consultants over the course of six calendar days.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Qingying Jie
Engineer
qingying@zellic.io ↗

Jinseo Kim
Engineer
jinseo@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

February 24, 2026	Start of primary review period
February 25, 2026	Kick-off call
March 3, 2026	End of primary review period

3. Detailed Findings

3.1. Users can avoid paying the fee when using `depositNative`

Target	Aori		
Category	Business Logic	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

This project connects users who want to swap their token to solvers. Solvers may charge some fee on the tokens being swapped as much as the order and the project specifies.

The `depositNative` function with a hook is newly added to the project in order to support swapping the native token to another token. Because native tokens cannot be transferred with allowance mechanisms (i.e., `approve` and `transferFrom`) — unlike the `deposit` function, which must be invoked by the solver with the signed order — the `depositNative` function can be invoked by the offerer of the order. A user is expected to invoke the `depositNative` function themselves with the native token and the hook information supplied through the off-chain infrastructure (if it exists). Due to this, users are allowed to invoke the `depositNative` function directly, unlike the `deposit` function, which can be only invoked by solvers.

It should be noted that the `depositNative` function does not verify that the hook information is validated by the solver. A user will be able to receive the hook information through the off-chain infrastructure and then submit the modified order that specifies a different solver.

Impact

A user can modify an order that involves swapping the native token to another token, avoiding paying the fee to the solver of the order.

Recommendations

Consider handling the business logic for native tokens through a wrapped native token. Alternatively, consider ensuring the solver is unchanged for the given order.

Remediation

This issue has been acknowledged by Aori, and a fix was implemented in commit `e48eaecb` ↗.

3.2. The function `_distributeOutputs` passes incorrect data when emitting the event `Deposit`

Target	AoriAtomicSwapLib		
Category	Coding Mistakes	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The third parameter of the event `Deposit` is the address of the token received from the source hook.

```
/**
 [...]
 * @param srcHookTokenOut The token received from the source hook (address(0)
    if no hook)
 * @param srcHookAmountOut The amount received from the source hook (0 if no
    hook)
 */
event Deposit(
    bytes32 indexed orderId,
    Order order,
    address indexed srcHookTokenOut,
    uint256 srcHookAmountOut
);
```

In the function `executeSwap`, after the function `executeHook` is executed, the `srcHookTokenOut` is `order.outputToken` and the `srcHookAmountOut` is `amountReceived`. When emitting the event `Deposit`, `preferredToken` is used as the `srcHookTokenOut`.

```
function executeSwap(
    bytes32 orderId,
    Order calldata order,
    SrcHook calldata hook,
    address solver
) external returns (uint256 amountReceived) {
    // [...]
    // Execute hook - converts input to output, validates minOutput
    amountReceived = HookUtils.executeHook(hook.hookAddress,
    hook.instructions, order.outputToken, minOutput);

    _distributeOutputs($, orderId, order, hook.preferredToken, solver,
```

```
    amountReceived);
}

function _distributeOutputs(
    AoriStorageData storage $,
    bytes32 orderId,
    Order calldata order,
    address preferredToken,
    address solver,
    uint256 amountReceived
) internal {
    // [...]
    emit IAori.Deposit(orderId, order, preferredToken, amountReceived);
    emit IAori.Fill(orderId, address(0), 0, 0);
    emit IAori.Settle(orderId, solver, SafeCast.toUint128(surplus),
    protocolFee, actualFeeRecipient, additionalFee);
}
```

Impact

If the values of `hook.preferredToken` and `order.outputToken` are different, the value of `srcHookTokenOut` does not correspond to the value of `srcHookAmountOut`. That is, the balance of the `preferredToken` does not change, which may cause confusion for users.

Recommendations

Consider changing `hook.preferredToken` to `order.outputToken`.

Remediation

This issue has been acknowledged by Aori, and a fix was implemented in commit `e48eaecb` ↗.

Aori provided the following response to this finding:

> Removed the `preferredToken` parameter entirely and emitting `order.outputToken` directly.
>
> Renamed Fill event fields to accurately reflect their generalized meaning across all order types — not just dstHook fills.
>
> Changed no-hook fill paths to emit the solver's actual cost (`outputToken`, `outputAmount`, 0) instead of zeros, so the Fill event always captures what the solver spent regardless of whether a hook was used.

These changes enable a single dollarized solver PnL formula across every execution path (atomic, no-hook, dstHook, cross-chain)

3.3. The solver set may differ across different chains

Target	Aori		
Category	Business Logic	**Severity**	Informational
Likelihood	Low	**Impact**	Informational

Description

The offerer can specify an authorized solver through the `options` field in the struct `Order`. When depositing tokens on the source chain, if specified, the contract checks whether the caller is the solver designated by the offerer. The same check is performed on the destination chain when filling the order.

```
struct Order {
    // [...]
    Options options;
}

struct Options {
    // [...]
    address solver;       // Authorized solver (address(0) = any whitelisted)
    uint16 slippageMbps;  // 0 = limit order, >0 = market order
}
```

Impact

If the authorized solvers are inconsistent across different chains, specifying certain solvers may cause the offerer's order to fail to settle.

Recommendations

Consider splitting the `solver` field into `srcSolver` and `dstSolver`.

Remediation

This issue has been acknowledged by Aori, and a fix was implemented in commit `e48eaecb` ↗.

4. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

4.1. Aori contract (version 0.4.0 upgrade)

The Aori contract is an intent settlement protocol designed to facilitate token exchanges across different blockchains between users and trusted solvers. Users can deposit tokens on a source chain with signed intent parameters, which solvers can fulfill on destination chains. The contract also supports hooks, which are external contract calls executed during the deposit or fill process. This feature enables more complex operations, such as swapping one token for another before a deposit or using a DEX to acquire the required output token during a fill. The contract charges a certain fee for each settled order.

Notable changes

The following outlines the notable changes in version 0.4.0:

- **Changes were made to the contract structure.** Version 0.3.1 uses a single file to hold the code of the main contract Aori, and all the libraries and library functions it relies on are contained in the file AoriUtils.sol. In the new version, part of the logic in the contract Aori has been moved into library functions. Different library functions are organized into multiple library files according to their functionality, such as AoriAdminLib.sol, AoriSettleLib.sol, TokenUtils.sol, and ValidationUtils.sol.
- **The regular contract was changed to an upgradable contract.** The UUPS pattern has been adopted, and the starting storage slot for the Aori contract's state variables was selected in accordance with ERC-7201.
- **Users are now allowed to deposit using signed Permit2 messages.** Two new functions were added: the function `depositWithPermit2` with a hook and the function `depositWithPermit2` without a hook. Users sign a Permit2 message that includes the order as witness data; while approving the contract Aori to transfer a specified amount of tokens, they also sign the order. These two functions are then called by the solver.
- **A dual-fee system was introduced.** Each settled order is charged a protocol fee and an additional fee. The protocol fee is set by the owner via the function `setProtocolFee`, while the additional fee (`feeMbps`) is included in the struct `Order` and can be signed and confirmed by the user.
- **New privileged functions were added to modify the newly introduced configuration.** New functions `claimProtocolFees`, `setProtocolFee`, `setProtocolTreasury`, and `setMaxFee` were added, mainly for collecting protocol fees and limiting the fee rate of additional fees.

- **Introduce a new function `depositNative` with a hook.** In version 0.3.1, only the function `depositNative` without a hook was available, which supported using the native token directly as the input token. The new function `depositNative` with a hook allows users to swap the native token for other tokens and use them as either the input token or the output token.
- **An `options` field was added to the struct `Order`.** The `options` field is of type struct `Options`, which includes the fields `feeMbps`, `feeRecipient`, `solver`, and `slippageMbps`. The offerer can sign to confirm these parameters. The `feeMbps` represents the proportion of the output-token amount paid as a fee to `feeRecipient`, while the `solver` field specifies that the caller must be a specific address. The `slippageMbps` represents the slippage tolerance that the offerer is willing to accept. In previous versions, the `solver` field lived in the struct `SrcHook` and could not be signed and confirmed by the offerer.

Minor changes

The following outlines minor changes in version 0.4.0:

- The compiler version has been upgraded from 0.8.28 to 0.8.34.
- The statement `require(condition, errorString)` has been changed to `if (!condition) revert customError()`.
- The function `emergencyWithdraw`, which extracts tokens from a user's balance, was renamed to `emergencyWithdrawFromBalance`.

5. Assessment Results

During our assessment on the scoped Aori 0.4.0 Upgrade contracts, we discovered three findings. No critical issues were found. One finding was of low impact and the other findings were informational in nature.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.